Docker

Best practices

Docker Security

Important security considerations:

- Scan images for vulnerabilities (docker scan)
- Use minimal base images (alpine, slim)
- Don't run containers as root
- Limit container capabilities
- Use read-only filesystems where possible
- Sign images with Docker Content Trust
- Apply resource limits
- Use security scanning tools (Trivy, Clair)
- Keep Docker engine and images updated
- Implement proper network segmentation

Docker Scout - Vulnerability Scanning

Docker directly provides a vulnerability scanner named Docker Scout in Docker Desktop.

```
# Docker Scout (integrated with Docker Desktop)
docker scout quickview image_name:tag

# Scan with detailed vulnerabilities
docker scout cves image_name:tag

# Generate SBOMs (Software Bill of Materials)
docker scout sbom image_name:tag
```

User Privileges and Security Contexts

Containers can be run as non-root by creating a user inside the image and using it with the USER directive. In alternative, it can be specified at runtime (note that the image has to be prepared to run as non-root anyway).

```
# Create a user in the Dockerfile
RUN groupadd -r appuser && useradd -r -g appuser appuser
USER appuser

# Or specify user at runtime
docker run --user 1000:1000 image_name
```

Limit container capabilities

Docker usually enable all capabilities.
It is a good practice to only enable needed ones:

```
# Drop all capabilities and add only needed ones
docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE image_name

# Run with read-only filesystem
docker run --read-only image_name
```

Container isolation

We can segment the network for each service:

```yaml
# Docker Compose example
services:
  frontend:
    networks:
      - frontend
  backend:
    networks:
      - frontend
      - backend
  database:
    networks:
      - backend

networks:
  frontend:
  backend:
    internal: true  # Not exposed to outside world
```

Resource limits

Each container can have limits on the available resources:

```
# Memory and CPU constraints
docker run --memory=2g --cpus=2 image_name

# With Compose
services:
  ml-api:
    deploy:
      resources:
        limits:
          cpus: '2'
          memory: 2G
```

Minimize images

- Use minimalistic base images, do not install unnecessary packages
  - E.g. Alpine, -slim, etc.

The benefits are:

  - Reduced size
  - Less attack surface

- Use `.dokerignore` to exclude non relevant files from build

- Use multi-stage builds
  - There's a build phase that uses a large base image to compile stuff and a runtime phase that copies the resulting app in a "lighter" base image

Deterministic images

- Ideally a Docker Image should be deterministic, i.e. should always produce the same result, but if we rely on latest versions…
  - Specify precisely the versions of the dependencies
    Pin base image versions, use:
    `FROM alpine:3.19`
    instead of
    `FROM alpine`
  - `FROM alpine:3.19@sha256:c5b1261d6d3e43071626931fc004f70149baeba2c8ec672bd4f27761f8e1ad6b`
    instead of
    `FROM alpine:3.19`
  - So that 3.19 tag refers to a specific version and not to 3.19.1, 3.19.2 etc. Get the sha256 from the repo.
  - Specify the versions of the packages in `requirements.txt`

Secure images

- Do not run “root” processes
- Manage stop signals to shutdown gracefully
  - `CMD ["app", "arg"]` (exec form) instead of `CMD app arg` (shell form) results in more responsiveness since we avoid that the running shell might handle differently the signals. The exec form gets a PID 1 and receives signals directly.
  - Same applies to using `docker exec –it CONTAINER my_command` instead of `docker exec –it CONTAINER sh –c “my_command”`
- Use ENV variables to allow personalization of secret keys, instead of storing them in .env files included in the image
  - It would be like storing secret keys in a Github repo !

Ephemeral containers

- It should be possible to destroy containers and rebuild them anew from images
- No data should be stored in containers, plan the architecture of the app accordingly
  - E.g. store in S3 bucket, on MySQL servers, etc.

- Many SaaS like Vercel, Heroku, etc. treat the Dockers as such

Makefiles for repetitive commands

- Makefiles can be used to handle repetitive commands, e.g. assign targets to build, start-up containers or docker-compose tasks, etc.

- Pros:
  - Automation: make COMMAND helps to run a certain command without having to remember all the parameters
  - Documentation: reading the Makefile targets we understand what to do
  - Consistency: the same commands are used by everybody
- Cons:
  - The Makefile syntax

Multi-stage Dockerfile

- Multi-stage builds help to create images:
  - Faster, parallelizing the build
  - Smaller, creating a final image with a smaller memory footprint from the artefacts created in the previous build stage

- A build stage is represented by a `FROM` instruction, this is extended adding a new `AS <stage-name>` to it.

- Use multiple FROM statements in the Dockerfile. Each FROM instruction can use a different base, and each of them begins a new stage of the build. Copy artifacts from one stage to another, leaving behind everything you don't want in the final image.

Multi-stage Dockerfile (reduce size)

```
# Stage 1: Build Environment
FROM builder-image AS build-stage
# Install build tools (e.g., Maven, Gradle)
# Copy source code
# Build commands (e.g., compile, package)

# Stage 2: Runtime environment
FROM runtime-image AS final-stage
#  Copy application artifacts from the build stage (e.g., JAR file)
COPY --from=build-stage /path/in/build/stage /path/to/place/in/final/stage
# Define runtime configuration (e.g., CMD, ENTRYPOINT)
```

Multi-stage Dockerfile (parallelize)

```
FROM xxxx AS base
# install build tools
# copy source code

FROM base AS build-client
# build client part of the system

FROM base AS build-server
# build server part of the system, should be independent from client

FROM xxxx/base
COPY --from=build-client /bin/client /bin/
COPY --from=build-server /bin/server /bin/
```

Multi-stage Dockerfile example

```
# Build stage
FROM node:16 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Production stage
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

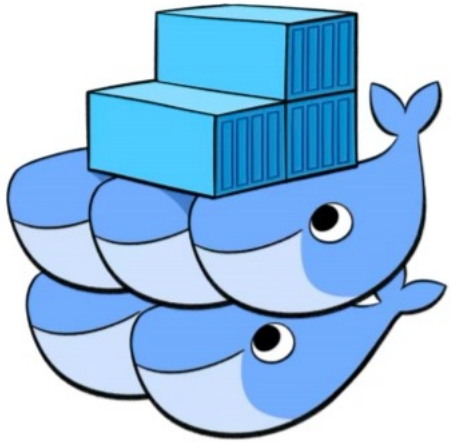
Exercise 1 - multistage

- Build both a single stage and a multiple stage Dockerfile in subproject 6_multistage.
- Check the differences in terms of image size
- Use `-f` to specify the multistage Dockerfile that has a non-default name

Container Orchestration

For managing containers at scale:

- Docker Swarm:
  - Built into Docker
  - Simpler setup for small to medium deployments
  - Integrated with Docker CLI



- Kubernetes:
  - Industry standard for container orchestration
  - More features and flexibility
  - Better for complex, large-scale deployments
  - Steeper learning curve

Monitoring Containers

- Tools and approaches for monitoring Docker containers:

- Built-in commands:

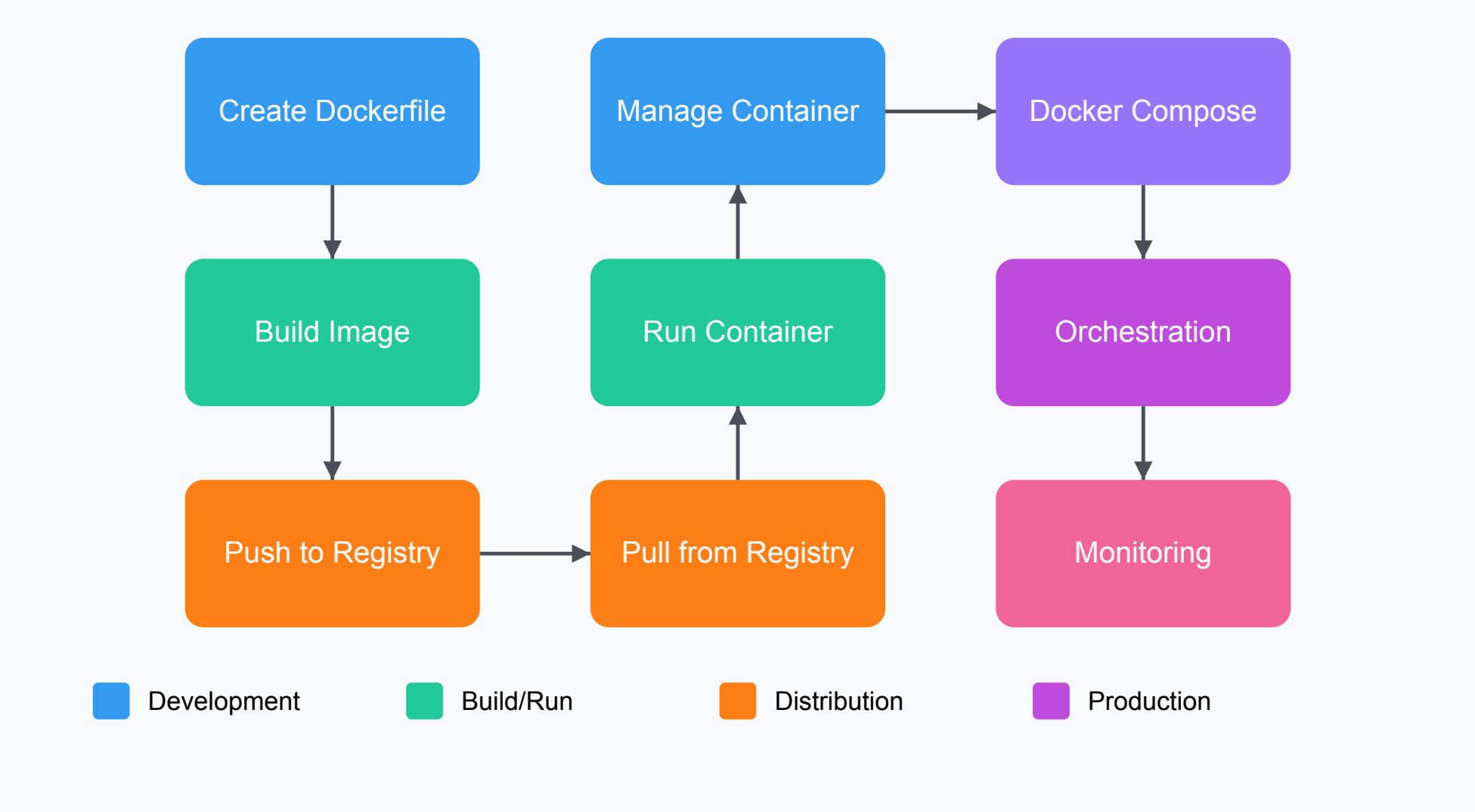
```
# Container resource usage
docker stats

# Container logs
docker logs container_id

# Container processes
docker top container_id
```

Monitoring Containers

- External tools:
  - Prometheus + Grafana
  - cAdvisor
  - Datadog
  - Portainer

- Metrics to monitor:
  - CPU and memory usage
  - Disk I/O
  - Network traffic
  - Container health
  - Application-specific metrics

Docker workflow



Docker for Development

Docker can also be used for development workflows.

- Benefits:
  - Consistent environments for all developers
  - "Works on my machine" problems eliminated
  - Easy onboarding of new team members
  - Simplified testing with different dependencies
  - Rapid development-to-production cycle
  - Isolation from host system

- Example workflow:
  - Development in containers with mounted source code
  - Hot reloading with mapped volumes
  - Debugging inside containers
  - Integration tests with Docker Compose

Docker Desktop Dev Environments

Dev environments are shareable, reproducible development environments, based on Docker Compose to share exact environments between team members.

- Configurable via devcontainer.json
- Integrates with VS Code, JetBrains IDEs
- Under heavy development

Benefits:

- Share exact environments between team members
- Onboard new developers in minutes
- Consistent tooling across the team
- Version control your development environment

Docker Desktop Dev Environments

- Can be specified in VSCode with a `devcontainer.json`

```
// devcontainer.json
{
  "name": "ML Development Environment",
  "dockerComposeFile": "docker-compose.yml",
  "service": "ml-workspace",
  "workspaceFolder": "/workspace",
  "extensions": [
    "ms-python.python",
    "ms-toolsai.jupyter",
    "ms-python.vscode-pylance"
  ],
  "settings": {
    "python.linting.enabled": true,
    "python.formatting.provider": "black"
  }
}
```